1. Опишите трехсторонний протокол распределения ключей между парами участников с использованием третьей стороны.

В сети с большим количеством пользователей, где каждый может обмениваться защищёнными сообщениями с любым другим, каждой паре нужен свой общий секретный ключ. При заранее предоставленных ключах в сети из n пользователей их будет n(n-1)/2, что при большом n очень много. Поэтому вводят доверенный центр распределения ключей. Каждый пользователь хранит только один долговременный ключ, общий с центром, а когда двум пользователям нужно связаться, центр генерирует для них сеансовый ключ. В итоге ключей нужно всего n. По такой схеме, например, работает протокол Нидхема-Шредера.

В протоколе участвуют три стороны:

- Алиса (А) - инициатор, запрашивает у центра ключ для связи с Бобом;
- Боб (Б) - второй участник, получает ключ через Алису и проверяет, что она им владеет;
- Центр (Ц) - доверенная третья сторона, хранит долговременные ключи всех пользователей, генерирует сеансовый ключ и распределяет его.

Обозначения: $k_{\text{А}}$ и $k_{\text{Б}}$ - долговременные ключи Алисы и Боба, общие с центром; $k_{\text{АБ}}$ - сеансовый ключ, который генерирует центр; $n_{\text{А}}$ и $n_{\text{Б}}$ - случайные числа; $E_k(\dots)$ - шифрование на ключе k.

0. *Отправитель → Получатель: <содержимое сообщения>. А и Б в сообщении - идентификаторы, по которым система их различает.*
1. А → Ц: А, Б, $n_{\text{А}}$. Алиса просит у центра ключ для связи с Бобом. Случайное число $n_{\text{А}}$ позволит ей потом отличить свежий ответ от повтора старого.

2. Ц → А: $E_{k_А}\left(n_А,\ Б,\ k_{АБ},\ E_{k_Б}(k_{АБ},\ А)\right)$. Центр генерирует $k_{АБ}$. Внутри ответа лежит билет для Боба $E_{k_Б}(k_{АБ},\ А)$, который Алиса прочитать не может. По $n_А$ Алиса проверяет, что ответ свежий, а по идентификатору Б - что ключ выдан именно для Боба. Без этого злоумышленник мог бы подменить в шаге 1 имя получателя на своё.
3. А → Б: $E_{k_Б}(k_{АБ},\ А)$. Алиса пересылает билет. Боб расшифровывает его своим ключом и узнаёт $k_{АБ}$ и то, что ключ предназначен для связи с Алисой.
4. Б → А: $E_{k_{АБ}}(n_Б)$. Боб пока точно не уверен в том, что общается с Алисой, дабы убедиться в этом, он отправляет случайное число $n_Б$ зашифрованное сеансовым ключом $k_{АБ}$.
5. А → Б: $E_{k_{АБ}}(n_Б - 1)$. Алиса доказывает Бобу, что владеет ключом $k_{АБ}$, отправляя зашифрованное сеансовым ключом ранее полученное от Боба случайное число $n_Б$, уменьшенное на единицу. Единицу вычитают, чтобы злоумышленник не мог просто вернуть Бобу его же сообщение 4, поскольку изменить число внутри можно, только расшифровав его.

В результате у Алисы и Боба появляется общий сеансовый ключ $k_{АБ}$, сгенерированный центром. Каждый из них убеждается в том, что второй тоже им владеет. Кроме них, этот ключ знает только центр.

2. Приведите формальные модели шифров.

Формальная модель нужна, чтобы описать шифр математически и исследовать его свойства строго, а не на отдельных примерах. Следуя К. Шеннону, выделяют две модели: алгебраическую и вероятностную.

Алгебраическая модель

Эта модель описывает устройство шифра, то есть что шифруется, на каких ключах и по каким правилам. С её помощью проверяют, что любой шифртекст расшифровывается однозначно.

Шифр задаётся набором

$$\Sigma_A = (X,\ K,\ Y,\ E,\ D),$$

компоненты которого:

- X - множество открытых текстов;
- K - множество ключей;
- Y - множество шифртекстов;
- $E = \{E_k : X \to Y\}_{k \in K}$ - правила зашифрования, по одному на каждый ключ;
- $D = \{D_k : E_k(X) \to X\}_{k \in K}$ - правила расшифрования.

Все множества конечны. Чтобы такой набор был шифром, должны выполняться два условия:

$$D_k\big(E_k(x)\big) = x \quad \text{для любых } x \in X,\ k \in K, \qquad\qquad Y = \bigcup_{k \in K} E_k(X).$$

Первое условие гарантирует однозначное расшифрование. Из него следует, что каждое E_k инъективно, то есть разные открытые тексты на одном ключе дают разные шифртексты, а при $X = Y$ отображение E_k биективно (такой шифр называют эндоморфным). Второе условие означает, что в Y нет лишних элементов, то есть каждый шифртекст получается из какого-то открытого текста на каком-то ключе.

В такой форме записываются шифры замены, перестановки, гаммирования. Однако о стойкости алгебраическая модель ничего не говорит, в ней нет понятия того, насколько вероятны те или иные тексты и ключи. Для этого нужна вторая модель.

Вероятностная модель

Эта модель позволяет оценивать стойкость шифра, то есть что противник может узнать об открытом тексте, перехватив шифртекст. На этой модели построена теория стойкости Шеннона.

Модель включает все компоненты алгебраической и два априорных распределения вероятностей:

$$\Sigma_B = \big(X,\ K,\ Y,\ E,\ D,\ P(X),\ P(K)\big),$$

- $P(X)$ - распределение на открытых текстах, показывающее, насколько вероятно то или иное сообщение (например, из-за статистики языка);
- $P(K)$ - распределение на ключах, обычно равновероятное, то есть $P(k) = 1/|K|$.

Ключ и открытый текст выбираются независимо. Тогда распределение на шифртекстах определяется однозначно:

$$P(y) = \sum_{(x,k)\,:\,E_k(x)=y} P\,(x)\,P(k).$$

В этой модели формулируются основные понятия теоретической стойкости:

- совершенная стойкость - $P(x \mid y) = P(x)$ для всех $x \in X$ и $y \in Y$ с $P(y) > 0$, то есть перехват шифртекста не меняет знаний противника об открытом тексте. Примером служит шифр Вернама при равновероятном ключе длины сообщения, используемом один раз;
- расстояние единственности - длина шифртекста, при которой ключ в среднем определяется однозначно.

3. Приведите принципы построения поточных систем шифрования.

В поточном шифре текст шифруется посимвольно, то есть по одному биту или байту. Каждый символ открытого текста складывается с очередным

символом гаммы - псевдослучайной последовательности, которую генератор вырабатывает по ключу. Чаще всего используется сложение по модулю 2:

$$y_i = x_i \oplus g_i, \qquad x_i = y_i \oplus g_i,$$

где x_i, g_i, y_i - i-е биты открытого текста, гаммы и шифртекста. Получатель с тем же ключом вырабатывает ту же гамму и повторным сложением восстанавливает открытый текст.

Поточная система состоит из двух блоков - генератора гаммы и шифрующего блока, который складывает гамму с текстом. Принципы их построения можно сгруппировать по тому, чего они позволяют добиться.

Принципы, обеспечивающие стойкость

1. Большой период гаммы. Нужен, чтобы гамма не повторялась. Если два текста зашифрованы одним и тем же отрезком гаммы, то при сложении шифртекстов гамма сокращается, и противник получает сумму открытых текстов, которую можно вскрыть статистически.
2. Близость гаммы к случайной последовательности. Нужна, чтобы шифртекст не сохранял статистику открытого текста и против шифра не работали статистические методы.
3. Отсутствие простых зависимостей между знаками гаммы. Нужно, чтобы по известной части гаммы нельзя было вычислить остальную.
4. Невозможность восстановить ключ по отрезку гаммы. Нужна потому, что при известном открытом тексте гамму легко получить, сложив его с шифртекстом. Ключ должен оставаться недоступным и в этом случае, дабы перехват части гаммы не раскрывал всю переписку.
5. Стойкость при перекрытиях гаммы, то есть при её повторном использовании. Для простого сложения по модулю 2 это требование невыполнимо (см. пункт 1), поэтому на практике повтор гаммы не допускают, вырабатывая её из ключа и уникальной синхропосылки. Один

ключ можно использовать многократно, а пару «ключ + синхропосылка» - никогда.

Принципы, обеспечивающие эффективность

6. Простые операции. Нужны для высокой скорости и простой аппаратной реализации, поэтому поточные шифры удобны там, где данные идут непрерывным потоком, например в каналах связи.
7. Универсальность. Шифр обрабатывает любые последовательности символов, поэтому одну систему можно применять к любым данным.

Принцип, обеспечивающий надёжность передачи

8. Синхронизация. Гамма у отправителя и получателя должна совпадать, иначе расшифрование нарушится. В синхронных шифрах гамма зависит только от ключа и синхропосылки, поэтому ошибки не размножаются, но потеря бита сбивает синхронизацию. В самосинхронизирующихся шифрах гамма зависит от предыдущих знаков шифртекста, поэтому синхронизация восстанавливается сама, но одна ошибка портит несколько следующих знаков (см. вопрос 5).

Принцип, обеспечивающий целостность

9. Дополнение имитовставкой. Гаммирование не защищает от подмены, ведь, изменив бит шифртекста, противник меняет тот же бит открытого текста. Поэтому для контроля целостности его дополняют имитовставкой.

Эти принципы необходимы, но не достаточны. Каждый из них закрывает отдельный класс атак, однако их выполнение ещё не гарантирует стойкости шифра в целом.

4. Приведите пример режима шифрования, в котором биективность всех шифрующих отображений базового режима не является необходимой для однозначного расшифрования криптограмм. Ответ обоснуйте.

Такой режим - гаммирование с обратной связью по шифртексту (CFB, в ГОСТ 28147-89 он называется «гаммирование с обратной связью»). В нём каждый блок шифруется по формуле

$$y_i = x_i \oplus E_k(y_{i-1}),$$

а расшифровывается по формуле

$$x_i = y_i \oplus E_k(y_{i-1}),$$

где x_i и y_i - блоки открытого текста и шифртекста, а в качестве y_0 берётся синхропосылка.

В базовом режиме простой замены блок шифруется как $y = E_k(x)$, и чтобы его расшифровать, нужно обратное отображение $x = E_k^{-1}(y)$. Поэтому там E_k обязано быть биекцией, иначе два разных блока могли бы дать один и тот же шифртекст, и получатель не смог бы понять, какой из них был отправлен.

В режиме CFB всё иначе. Отображение E_k к открытому тексту вообще не применяется, оно только вырабатывает гамму из предыдущего блока шифртекста. Обратное отображение не нужно ни при шифровании, ни при расшифровании, обе стороны используют одно и то же прямое E_k. Поэтому от него требуется только детерминированность, то есть чтобы при одинаковом входе получался одинаковый результат. Это выполняется для любой функции, даже необратимой.

При этом расшифрование остаётся однозначным. Получатель знает ключ и уже принял предыдущий блок y_{i-1} (для первого блока это синхропосылка), значит, он может сам вычислить ту же гамму $g_i = E_k(y_{i-1})$, что и отправитель.

Остаётся сложить её с принятым блоком, и так как $g_i \oplus g_i = 0$, получится открытый текст:

$$y_i \oplus g_i = x_i \oplus g_i \oplus g_i = x_i.$$

Выходит, однозначность обеспечивает обратимость сложения по модулю 2, а не обратимость E_k. Точно так же устроены режим гаммирования и режим обратной связи по выходу (OFB). Правда, на стойкость небиективность всё же влияет плохо, ведь разные входы могут дать одинаковую гамму, но для однозначного расшифрования это неважно.

5. Сколько 64-битовых блоков открытого текста искажается при расшифровании криптограммы симметричного блочного шифра, если в шифрованном тексте исказился один бит и используется режим гаммирования с обратной связью по шифртексту, в котором блоки открытого и шифрованного текста имеют длину 8 бит? Ответ обоснуйте.

Блочный шифр здесь работает с 64-битовыми блоками, а текст шифруется по 8 бит, то есть по байту. Для этого используется 64-битовый регистр, в котором всегда лежат последние 8 байтов шифртекста (в самом начале - синхропосылка). Чтобы зашифровать очередной байт, регистр шифруется блочным шифром, из результата берутся старшие 8 бит, и они служат гаммой. Байт текста складывается с гаммой, а получившийся байт шифртекста вдвигается в регистр. Расшифрование идёт так же, только с гаммой складывается принятый байт шифртекста:

$$x_i = y_i \oplus \mathrm{MSB}_8\big(E_k(y_{i-8}, \ldots, y_{i-1})\big).$$

Пусть в байте шифртекста y_i исказился один бит. Тогда при расшифровании:

- в x_i будет искажён только этот же бит, потому что гамма для него считалась по регистру, в который y_i ещё не попал;

- следующие 8 байтов $x_{i+1}, \dots, x_{i+8}$ будут искажены непредсказуемо, потому что всё это время искажённый y_i лежит в регистре и портит гамму. В регистре помещается $64/8 = 8$ байтов, поэтому таких байтов именно восемь;
- начиная с x_{i+9} всё расшифровывается правильно, так как y_i уже вытеснен из регистра.

Итого искажаются $1 + 8 = 9$ байтов подряд.

Теперь посмотрим, сколько это 64-битовых блоков открытого текста. В каждом блоке 8 байтов, поэтому 9 байтов подряд в один блок не влезут, а чтобы задеть три блока, нужно хотя бы 10 байтов. Значит, ошибка всегда задевает ровно два соседних блока. Например, если исказился первый байт блока, то испорчен весь этот блок и первый байт следующего, а если последний, то последний байт этого блока и весь следующий блок.

Ответ: искажаются 2 блока открытого текста по 64 бита (9 байтов подряд).

6. Однонаправленная хэш-функция принимает значения из множества двоичных n-мерных векторов. При каком n случайная пара сообщений составляет коллизию с вероятностью, не большей 10^{-9}? Обязательно должен быть приведен ход решения.

Коллизия - это когда у двух разных сообщений совпадают хэш-значения. Для решения воспользуемся моделью парадокса дней рождения. Будем считать, что хэш-функция принимает 2^n значений, и для разных сообщений они независимы и равновероятны. Тогда вероятность того, что среди k сообщений хотя бы у двух хэши совпадут, равна

$$P(k) = 1 - \prod_{i=1}^{k-1} \left(1 - \frac{i}{2^n}\right).$$

Произведение в этой формуле показывает вероятность того, что все хэши окажутся разными, то есть второй не совпадёт с первым, третий ни с одним из первых двух и так далее. Коллизия - противоположное событие, поэтому произведение вычитается из единицы.

У нас пара сообщений, то есть $k = 2$, и в произведении остаётся один множитель:

$$P(2) = 1 - \left(1 - \frac{1}{2^n}\right) = \frac{1}{2^n}.$$

По условию эта вероятность не больше 10^{-9}:

$$\frac{1}{2^n} \leq 10^{-9}.$$

Обе части положительны, поэтому можно перевернуть дроби, поменяв знак неравенства:

$$2^n \geq 10^9.$$

Логарифмируем по основанию 2:

$$n \geq \log_2 10^9 = 9\log_2 10.$$

Считаем $\log_2 10$ через натуральные логарифмы:

$$\log_2 10 = \frac{\ln 10}{\ln 2} \approx \frac{2{,}302585}{0{,}693147} \approx 3{,}321928,$$

$$n \geq 9 \cdot 3{,}321928 \approx 29{,}897.$$

Поскольку n целое, получаем $n \geq 30$.

Проверим. При $n = 30$ вероятность равна $1/2^{30} = 1/1\,073\,741\,824 \approx 9{,}31 \cdot 10^{-10}$, это меньше 10^{-9}. А при $n = 29$ она равна $1/2^{29} = 1/536\,870\,912 \approx 1{,}86 \cdot 10^{-9}$, это уже больше. Значит, 30 - минимальное подходящее значение.

Ответ: $n \geq 30$.